

Docker

Practical tasks

Practical task 1

Follow the steps below and practice with Dockerfile and docker-compose configuration.

- Fork  [GitHub - spring-projects/spring-petclinic: A sample Spring-based application](#)
- Create Dockerfile for Spring-petclinic application using pre-built artifact
 - Build application outside of container
 - Copy artifact from target folder into image and make it work inside container
- Create multi-stage Dockerfile for Spring-petclinic application
 - Application should be built as part of first stage
 - Final image should contain only required files and based on minimal possible base image
- Create docker-compose configuration that will automatically start multiple containers
 - Run two containers: application + DB
 - Provide credentials as environment variables, so DB image can be configured with custom credentials and application can connect to DB automatically

Practical task 2

Follow the steps below to practice with using remote repositories for storing Docker images and executing security scans:

Prerequisites:

- Use images that were built during Docker practice task
- Get/check access to corporate AWS account and read basic instructions
 - [FAQ for interns](#)
 - [GD AWS cloud for beginners](#)
 - [Tagging policy for Grid Dynamics AWS accounts](#)
- Read documentation about Docker repositories in Nexus
 - [Using Nexus 3 as Your Repository – Part 3: Docker Images](#)
- Read documentation about AWS ECR
 - [Getting started with Amazon ECR using the AWS Management Console - Amazon ECR](#)
 - [Using Amazon ECR with the AWS CLI - Amazon ECR](#)

Task:

- Uploading docker images to Nexus and ECR
 - Create and configure docker repository in Nexus
 - Upload spring-petclinic image to Nexus
 - Create and configure repository in ECR
 - Upload spring-petclinic image to ECR
- Perform security scan for uploaded images in ECR
 - Get scan report
 - Analyze results

There might be (or not) security vulnerabilities that need to be addressed. Discussion about possible workarounds to get rid of them and overall analysis of reports should be held.

Practical task 1:

- Create Dockerfile for Spring-petclinic application using pre-built artifact
 - Build application outside of container
 - Copy artifact from target folder into image and make it work inside container

Step 1: Fork & Clone Repo**Fork:**

<https://github.com/spring-projects/spring-petclinic>

Then:

```
rpavani@C19536 % git clone https://github.com/rpavani-png/spring-petclinic.git
rpavani@C19536 % cd spring-petclinic
```

Step 2: Build Application (Outside Container)

Spring Boot uses Maven:

```
./mvnw clean package -DskipTests
```

After build:

```
target/spring-petclinic-*.jar
```

Step 3: Simple Dockerfile (Using Pre-built Artifact)

```
FROM eclipse-temurin:17-jdk-jammy
```

```
WORKDIR /app
```

```
COPY target/spring-petclinic-*.jar app.jar
```

```
EXPOSE 8080
```

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

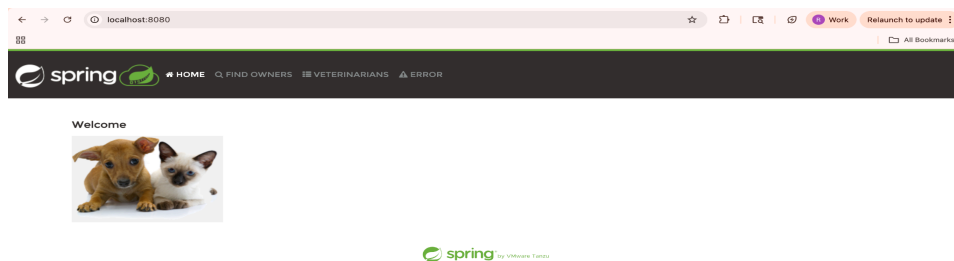
Step 4: Build & Run Image

```
docker build -f Dockerfile.basic -t petclinic:basic .  
docker run -p 8080:8080 petclinic:basic
```

Check:

<http://localhost:8080>

Output:



- Create multi-stage Dockerfile for Spring-petclinic application
 - Application should be built as part of first stage
 - Final image should contain only required files and based on minimal possible base image

Step 5: Multi-stage Dockerfile (BEST PRACTICE)

Now we build inside Docker (clean + production-ready image)

```
# Stage 1: Build
```

```
FROM maven:3.9.9-eclipse-temurin-17 AS builder

WORKDIR /build

COPY . .

RUN mvn clean package -DskipTests

# Stage 2: Runtime

FROM eclipse-temurin:17-jre-jammy

WORKDIR /app

COPY --from=builder /build/target/spring-petclinic-*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Step 6: Build Multi-stage Image

```
docker build -f Dockerfile.multi -t petclinic:multi .
```

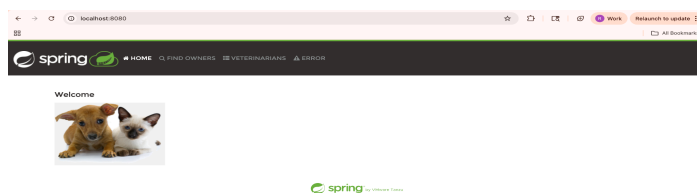
```
docker run -p 8080:8080 petclinic:multi
```

Verify Application

Open:

<http://localhost:8080>

Output:



- Create docker-compose configuration that will automatically start multiple containers
 - Run two containers: application + DB
 - Provide credentials as environment variables, so DB image can be configured with custom credentials and application can connect to DB automatically

Step 7: Docker Compose (App + DB)

Spring Petclinic supports MySQL profiles.

Create `docker-compose.yml`:

```
services:
  db:
    image: mysql:8
    container_name: petclinic-db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h localhost -u root -p${MYSQL_ROOT_PASSWORD}"]
      interval: 10s
      timeout: 5s
      retries: 10

  app:
    build:
      context: .
      dockerfile: Dockerfile.multi
    container_name: petclinic-app
    restart: always
    depends_on:
      db:
        condition: service_healthy
    ports:
      - "8080:8080"
    environment:
      SPRING_PROFILES_ACTIVE: ${SPRING_PROFILES_ACTIVE}
      SPRING_DATASOURCE_URL: ${SPRING_DATASOURCE_URL}
      SPRING_DATASOURCE_USERNAME: ${SPRING_DATASOURCE_USERNAME}
      SPRING_DATASOURCE_PASSWORD: ${SPRING_DATASOURCE_PASSWORD}
```

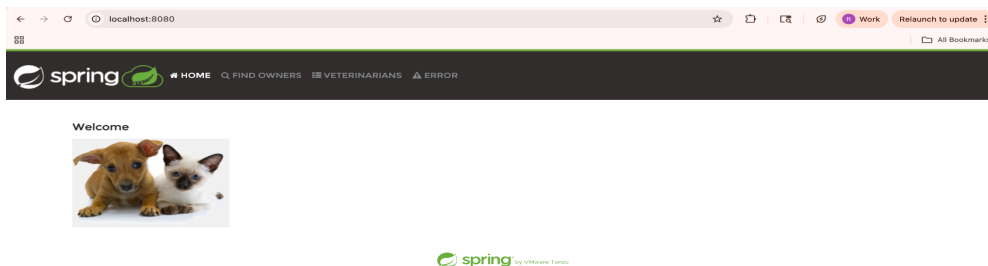
Step 8: Run Everything

`docker-compose up --build`

App:

<http://localhost:8080>

Output:



Practical task 2:

- Uploading docker images to Nexus and ECR
 - Create and configure docker repository in Nexus
 - Upload spring-petclinic image to Nexus
 - Create and configure repository in ECR
 - Upload spring-petclinic image to ECR
- Perform security scan for uploaded images in ECR
 - Get scan report
 - Analyze results

PART 1 — Upload Docker Image to Nexus

1. Create Docker Repository in Sonatype Nexus Repository Manager

1. Open Nexus UI → <http://<your-nexus-ip>:8081>
2. Go to:

Settings → Repositories → Create repository

3. Choose:

docker (hosted)

4. Configure:
 - Name: **docker-hosted**
 - HTTP Port: **8082** (or any free port)
 - Enable Docker V1 API: (keep disabled)
 - Deployment policy: Allow redeploy
 5. Save
-

2. Enable Docker Login to Nexus

Edit Docker daemon config:

```
sudo vi /etc/docker/daemon.json
```

Add:

```
{  
  "insecure-registries": ["<your-nexus-ip>:8082"]  
}
```

Restart Docker:

```
sudo systemctl restart docker
```

3. Login to Nexus Docker Registry

```
docker login <your-nexus-ip>:8082
```

Use:

- Username: admin
 - Password: from Nexus
-

4. Tag Spring Petclinic Image


```

=> => exporting layers                                0.0s
=> => exporting manifest sha256:50403d7732ebfaf5ff1c9a2b05c4a2a709c482a57abf2513c480f7a299a49 0.0s
=> => exporting config sha256:59ab53d4c931a5b6594b1b4f7af2efb67bb4a15ecfb1f90dcc19d4bc56a4172 0.0s
=> => exporting attestation manifest sha256:e912d1f09f8d0a99f6990e8c9615292f21491912fc90f2a34 0.0s
=> => exporting manifest list sha256:32bd44f8f6033926f8bf8c678b669d0b54ec30dd600e84c8b428c541 0.0s
=> => naming to docker.io/library/spring-petclinic-app:latest 0.0s
=> => unpacking to docker.io/library/spring-petclinic-app:latest 0.0s
=> resolving provenance for metadata file 0.0s
[+] up 4/4
✓ Image spring-petclinic-app Built 1.6s
✓ Network spring-petclinic_default Created 0.0s
✓ Container petclinic-db Created 0.0s
✓ Container petclinic-app Created 0.0s
Attaching to petclinic-app, petclinic-db
petclinic-db | 2026-04-29 06:29:46+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.9-1.el9
started.
Container petclinic-db Waiting
petclinic-db | 2026-04-29 06:29:46+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
petclinic-db | 2026-04-29 06:29:46+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.9-1.el9
started.
petclinic-db | 2026-04-29 06:29:46+00:00 [Note] [Entrypoint]: Initializing database files
petclinic-db | 2026-04-29T06:29:46.595571Z 0 [System] [MY-015017] [Server] MySQL Server Initialization - start.
petclinic-db | 2026-04-29T06:29:46.596116Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.4.9)
initializing of server in progress as process 79
petclinic-db | 2026-04-29T06:29:46.599427Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
petclinic-db | 2026-04-29T06:29:46.696127Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
petclinic-db | 2026-04-29T06:29:47.111673Z 6 [Warning] [MY-010453] [Server] root@localhost is created with an
empty password ! Please consider switching off the --initialize-insecure option.
petclinic-db | 2026-04-29T06:29:48.504832Z 0 [System] [MY-015018] [Server] MySQL Server Initialization - end.
petclinic-db | 2026-04-29 06:29:48+00:00 [Note] [Entrypoint]: Database files initialized
petclinic-db | 2026-04-29 06:29:48+00:00 [Note] [Entrypoint]: Starting temporary server
petclinic-db | 2026-04-29T06:29:48.544196Z 0 [System] [MY-015015] [Server] MySQL Server - start.
petclinic-db | 2026-04-29T06:29:48.693585Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.4.9)
starting as process 122
petclinic-db | 2026-04-29T06:29:48.699047Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
petclinic-db | 2026-04-29T06:29:48.780761Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
petclinic-db | 2026-04-29T06:29:48.902138Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self
signed.
petclinic-db | 2026-04-29T06:29:48.902157Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to
support TLS. Encrypted connections are now supported for this channel.
petclinic-db | 2026-04-29T06:29:48.903324Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file:
Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different directory.
petclinic-db | 2026-04-29T06:29:48.909204Z 0 [System] [MY-011323] [Server] X Plugin ready for connections.
Socket: /var/run/mysqld/mysqlx.sock
petclinic-db | 2026-04-29T06:29:48.909303Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for
connections. Version: '8.4.9' socket: '/var/run/mysqld/mysqld.sock' port: 0 MySQL Community Server - GPL.
petclinic-db | 2026-04-29 06:29:48+00:00 [Note] [Entrypoint]: Temporary server started.
petclinic-db | '/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
petclinic-db | Warning: Unable to load '/usr/share/zoneinfo/iso3166.tab' as time zone. Skipping it.
petclinic-db | Warning: Unable to load '/usr/share/zoneinfo/leap-seconds.list' as time zone. Skipping it.
petclinic-db | Warning: Unable to load '/usr/share/zoneinfo/leapseconds' as time zone. Skipping it.
petclinic-db | Warning: Unable to load '/usr/share/zoneinfo/tzdata.zi' as time zone. Skipping it.
petclinic-db | Warning: Unable to load '/usr/share/zoneinfo/zone.tab' as time zone. Skipping it.

```

[illegible]

```
petclinic-app | 2026-04-29T06:29:57.779Z INFO 1 --- [main] .s.d.r.c.RepositoryConfigurationDelegate :  
Finished Spring Data repository scanning in 16 ms. Found 3 JPA repository interfaces.  
petclinic-app | 2026-04-29T06:29:58.043Z INFO 1 --- [main] o.s.boot.tomcat.TomcatWebServer :  
Tomcat initialized with port 8080 (http)  
petclinic-app | 2026-04-29T06:29:58.048Z INFO 1 --- [main] o.apache.catalina.core.StandardService :  
Starting service [Tomcat]  
petclinic-app | 2026-04-29T06:29:58.048Z INFO 1 --- [main] o.apache.catalina.core.StandardEngine :  
Starting Servlet engine: [Apache Tomcat/11.0.18]  
petclinic-app | 2026-04-29T06:29:58.061Z INFO 1 --- [main] b.w.c.s.WebApplicationContextInitializer :  
Root WebApplicationContext: initialization completed in 638 ms  
petclinic-app | 2026-04-29T06:29:58.151Z INFO 1 --- [main] com.zaxxer.hikari.HikariDataSource :  
HikariPool-1 - Starting...  
petclinic-app | 2026-04-29T06:29:58.299Z INFO 1 --- [main] com.zaxxer.hikari.pool.HikariPool :  
HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@12e007be  
petclinic-app | 2026-04-29T06:29:58.300Z INFO 1 --- [main] com.zaxxer.hikari.HikariDataSource :  
HikariPool-1 - Start completed.  
petclinic-app | 2026-04-29T06:29:58.422Z INFO 1 --- [main] org.hibernate.orm.jpa :  
HHH008540: Processing PersistenceUnitInfo [name: default]  
petclinic-app | 2026-04-29T06:29:58.444Z INFO 1 --- [main] org.hibernate.orm.core :  
HHH000001: Hibernate ORM core version 7.2.4.Final  
petclinic-app | 2026-04-29T06:29:58.618Z INFO 1 --- [main] o.s.o.j.p.SpringPersistenceUnitInfo : No  
LoadTimeWeaver setup: ignoring JPA class transformer  
petclinic-app | 2026-04-29T06:29:58.660Z INFO 1 --- [main] org.hibernate.orm.connections.pooling :  
HHH10001005: Database info:  
petclinic-app | Database JDBC URL [jdbc:mysql://db:3306/petclinic]  
petclinic-app | Database driver: MySQL Connector/J  
petclinic-app | Database dialect: MySQLDialect  
petclinic-app | Database version: 8.4.9  
petclinic-app | Default catalog/schema: petclinic/undefined  
petclinic-app | Autocommit mode: undefined/unknown  
petclinic-app | Isolation level: REPEATABLE_READ [default REPEATABLE_READ]  
petclinic-app | JDBC fetch size: none  
petclinic-app | Pool: DataSourceConnectionProvider  
petclinic-app | Minimum pool size: undefined/unknown  
petclinic-app | Maximum pool size: undefined/unknown  
petclinic-app | 2026-04-29T06:29:59.033Z INFO 1 --- [main] org.hibernate.orm.core :  
HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)  
petclinic-app | 2026-04-29T06:29:59.036Z INFO 1 --- [main] j.LocalContainerEntityManagerFactoryBean :  
Initialized JPA EntityManagerFactory for persistence unit 'default'  
petclinic-app | 2026-04-29T06:29:59.070Z INFO 1 --- [main] o.s.d.j.r.query.QueryEnhancerFactories :  
Hibernate is in classpath; If applicable, HQL parser will be used.  
petclinic-app | 2026-04-29T06:29:59.549Z INFO 1 --- [main] o.s.b.a.e.web.EndpointLinksResolver :  
Exposing 13 endpoints beneath base path '/actuator'  
petclinic-app | 2026-04-29T06:29:59.584Z INFO 1 --- [main] o.s.boot.tomcat.TomcatWebServer :  
Tomcat started on port 8080 (http) with context path '/'  
petclinic-app | 2026-04-29T06:29:59.588Z INFO 1 --- [main] o.s.s.petclinic.PetClinicApplication : Started  
PetClinicApplication in 2.374 seconds (process running for 2.586)  
petclinic-app | 2026-04-29T06:30:01.144Z INFO 1 --- [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] :  
Initializing Spring DispatcherServlet 'dispatcherServlet'  
petclinic-app | 2026-04-29T06:30:01.144Z INFO 1 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet :  
Initializing Servlet 'dispatcherServlet'  
petclinic-app | 2026-04-29T06:30:01.145Z INFO 1 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet :  
Completed initialization in 1 ms
```

```

Gracefully Stopping... press Ctrl+C again to force
Container petclinic-app Stopping
petclinic-app | 2026-04-29T12:19:10.940Z INFO 1 --- [ionShutdownHook] o.s.boot.tomcat.GracefulShutdown
: Commencing graceful shutdown. Waiting for active requests to complete
petclinic-app | 2026-04-29T12:19:10.946Z INFO 1 --- [tomcat-shutdown] o.s.boot.tomcat.GracefulShutdown :
Graceful shutdown complete
petclinic-app | 2026-04-29T12:19:10.955Z INFO 1 --- [ionShutdownHook]
j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
petclinic-app | 2026-04-29T12:19:10.960Z INFO 1 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource
: HikariPool-1 - Shutdown initiated...
petclinic-app | 2026-04-29T12:19:10.965Z INFO 1 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource
: HikariPool-1 - Shutdown completed.
Container petclinic-app Stopped
Container petclinic-db Stopping
petclinic-app exited with code 143
petclinic-db | 2026-04-29T12:19:11.148617Z 0 [System] [MY-013172] [Server] Received SHUTDOWN from user
<via user signal>. Shutting down mysqld (Version: 8.4.9).
petclinic-db | 2026-04-29T12:19:11.868503Z 0 [System] [MY-010910] [Server] /usr/sbin/mysqld: Shutdown
complete (mysqld 8.4.9) MySQL Community Server - GPL.
petclinic-db | 2026-04-29T12:19:11.868527Z 0 [System] [MY-015016] [Server] MySQL Server - end.
Container petclinic-db Stopped
petclinic-db exited with code 0
rpavani@C19536 spring-petclinic % docker exec -it nexus cat /nexus-data/admin.password
5027d414-6492-450c-8b81-26ea0dbd1488%

rpavani@C19536 spring-petclinic % docker ps -a | grep nexus
b3caa1260a08 sonatype/nexus3 "/opt/sonatype/nexus..." 21 hours ago Up 37 seconds
0.0.0.0:8081-8083->8081-8083/tcp, [::]:8081-8083->8081-8083/tcp nexus

rpavani@C19536 spring-petclinic % docker login localhost:8083
Username: admin
Password:
Login Succeeded

rpavani@C19536 spring-petclinic % docker tag petclinic:multi localhost:8083/petclinic:latest

rpavani@C19536 spring-petclinic % docker push localhost:8083/petclinic:latest
The push refers to repository [localhost:8083/petclinic]
6c22b9a41346: Pushed
936158611fed: Pushed
6edbc812af48: Pushed
5efd89e8c4ee: Pushed
c36a0d5f3c86: Pushed
9d98330033bf: Pushed
2db291c6cfd3: Pushed
6b9e8b1b8a65: Pushed
latest: digest: sha256:473fc3ec56c5b2318866939eedda520832b110aa953aef8e58c18b47099b3d68 size: 856

rpavani@C19536 spring-petclinic % docker pull localhost:8083/petclinic:latest
latest: Pulling from petclinic
Digest: sha256:473fc3ec56c5b2318866939eedda520832b110aa953aef8e58c18b47099b3d68
Status: Image is up to date for localhost:8083/petclinic:latest

```

```
localhost:8083/petclinic:latest
```

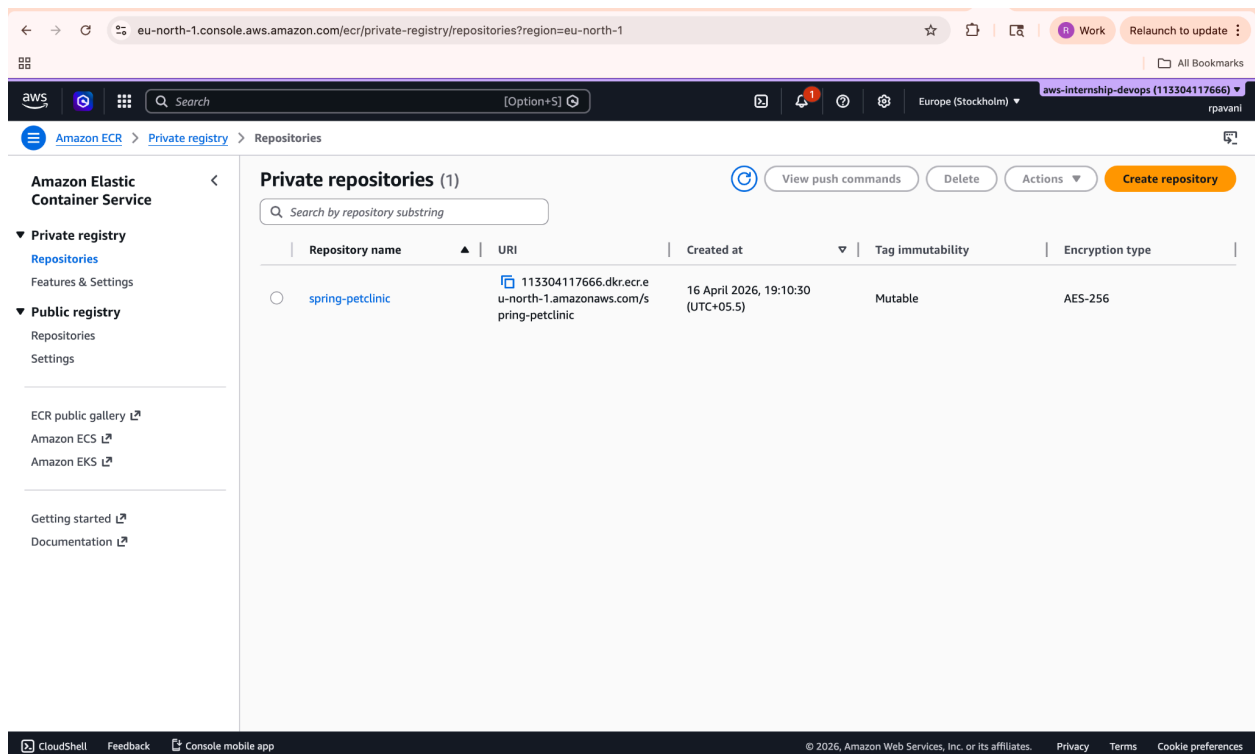
PART 2 — Upload to AWS ECR

Using Amazon Web Services ECR.

1. Create Repository in Amazon Elastic Container Registry

Using CLI:

```
aws ecr create-repository \
--repository-name spring-petclinic \
--region <your-region>
```



2. Authenticate Docker to ECR

```
aws ecr get-login-password --region <region> \
| docker login --username AWS --password-stdin <account-id>.dkr.ecr.<region>.amazonaws.com
```

```
rpavani@C19536 spring-petclinic % brew install awscli
```

```
==> Auto-updating Homebrew...
```

```
Adjust how often this is run with `HOMEBREW_AUTO_UPDATE_SECS` or disable with
```

```
`HOMEBREW_NO_AUTO_UPDATE=1`. Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
```

```
==> Downloading
```

```
https://ghcr.io/v2/homebrew/core/portable-ruby/blobs/sha256:8ad98d9cf15477d1a43f4012cfd5eff5c271398bca1e2724b55bd4944a0bb1d4
```

```
#####  
### 100.0%
```

```
==> Pouring portable-ruby-4.0.3.arm64_big_sur.bottle.tar.gz
```

```
==> Auto-updated Homebrew!
```

```
Updated 3 taps (hashicorp/tap, homebrew/core and homebrew/cask).
```

```
==> New Formulae
```

```
alevin-fry: Efficient and flexible tool for processing single-cell sequencing data
```

```
barman: Backup and Recovery Manager for PostgreSQL
```

```
defuddle: Extract article content and metadata from web pages
```

```
erfa: Essential Routines for Fundamental Astronomy
```

```
hf-mount: Mount Hugging Face Buckets and repos as local filesystems
```

```
kubernetes-cli@1.35: Kubernetes command-line interface
```

```
libcharset: Mozilla's Universal Charset Detector C/C++ API
```

```
mcp-remote: Remote proxy for Model Context Protocol with OAuth support
```

```
oarfish: Long read RNA-seq quantification
```

```
pomerium: Identity and context-aware access proxy
```

```
rabbitmqadmin: Command-line tool for RabbitMQ that uses the HTTP API
```

```
sdl3_mixer: Sample multi-channel audio mixer library
```

```
zfp: Compressed numerical arrays that support high-speed random access
```

==> New Casks

gopher64: N64 emulator

ipplay: Multimedia player

openusage: AI usage tracker for Cursor, Claude Code, Codex, Copilot and more

rustcast: Application and utility launcher

server-box: App for monitoring server status with SSH terminal, SFTP, Container management

sharefile: Client for the Progress ShareFile storage service

tolaria: Markdown knowledgebase manager

ua-connect: Software installer and device manager for Universal Audio products

wox: Launcher tool

You have 10 outdated formulae and 1 outdated cask installed.

==> Fetching downloads for: awscli

✓ Bottle Manifest awscli (2.34.39)	Downloaded 22.4KB/ 22.4KB
✓ Bottle Manifest python@3.14 (3.14.4_1)	Downloaded 31.1KB/ 31.1KB
✓ Bottle awscli (2.34.39)	Downloaded 22.9MB/ 22.9MB
✓ Bottle python@3.14 (3.14.4_1)	Downloaded 19.2MB/ 19.2MB

==> Installing awscli dependency: python@3.14

==> Pouring python@3.14--3.14.4_1.arm64_tahoe.bottle.tar.gz

🍺 /opt/homebrew/Cellar/python@3.14/3.14.4_1: 3,745 files, 75.5MB

==> Pouring awscli--2.34.39.arm64_tahoe.bottle.tar.gz

==> Caveats

The "examples" directory has been installed to:

/opt/homebrew/share/awscli/examples

==> Summary

🍺 /opt/homebrew/Cellar/awscli/2.34.39: 14,522 files, 156MB

==> Running `brew cleanup awscli` ...

Disable this behaviour by setting `HOMEBREW\_NO\_INSTALL\_CLEANUP=1`.

Hide these hints with `HOMEBREW\_NO\_ENV\_HINTS=1` (see `man brew`).

==> Caveats

zsh completions and functions have been installed to:

/opt/homebrew/share/zsh/site-functions

==> awscli

The "examples" directory has been installed to:

/opt/homebrew/share/awscli/examples

rpavani@C19536 spring-petclinic % aws configure

AWS Access Key ID [*****7666]: xxxxxxxxxxxxxxxxxxxxxxxxxxxx

AWS Secret Access Key [*****2301]: xxxxxxxxxxxxxxxxxxxxxxxxxxxx

Default region name [eu-north-1]:

Default output format [None]:

rpavani@C19536 spring-petclinic % aws ecr get-login-password --region eu-north-1 \

| docker login --username AWS --password-stdin 113304117666.dkr.ecr.eu-north-1.amazonaws.com

Login Succeeded

3. Tag Image for ECR

docker tag petclinic:basic <account-id>.dkr.ecr.<region>.amazonaws.com/spring-petclinic:latest

4. Push Image to ECR

docker push <account-id>.dkr.ecr.<region>.amazonaws.com/spring-petclinic:latest

Verify in AWS Console → ECR → Images

```
rpavani@C19536 spring-petclinic % docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
localhost:8083/petclinic:latest	473fc3ec56c5	494MB	153MB	U
mysql:8	43bd9764df60	1.11GB	249MB	U
nexus-url/repository/petclinic:latest	473fc3ec56c5	494MB	153MB	U
petclinic:basic	c91eced3dd13	756MB	256MB	U
petclinic:multi	473fc3ec56c5	494MB	153MB	U
sonatype/nexus3:latest	5b18c40967ea	1.07GB	437MB	U
spring-petclinic-app:latest	32bd44f8f603	494MB	153MB	U

```
rpavani@C19536 spring-petclinic % docker tag petclinic:basic
113304117666.dkr.ecr.eu-north-1.amazonaws.com/spring-petclinic:latest
```

```
rpavani@C19536 spring-petclinic % docker tag petclinic:multi
113304117666.dkr.ecr.eu-north-1.amazonaws.com/spring-petclinic:multi
```

```
docker push 113304117666.dkr.ecr.eu-north-1.amazonaws.com/spring-petclinic:multi
```

The push refers to repository [113304117666.dkr.ecr.eu-north-1.amazonaws.com/spring-petclinic]

c36a0d5f3c86: Pushed

6edbc812af48: Layer already exists

9d98330033bf: Layer already exists

936158611fed: Layer already exists

2db291c6cfd3: Layer already exists

6b9e8b1b8a65: Layer already exists

6c22b9a41346: Pushed

5efd89e8c4ee: Pushed

multi: digest: sha256:473fc3ec56c5b2318866939eedda520832b110aa953aef8e58c18b47099b3d68 size: 856

eu-north-1.console.aws.amazon.com/ecr/repositories/private/113304117666/spring-petclinic/_details?region=eu-north-1

Amazon Elastic Container Service

▼ Private registry

Repositories

spring-petclinic

Features & Settings

▼ Public registry

Repositories

Settings

ECR public gallery

Amazon ECS

Amazon EKS

Getting started

Documentation

spring-petclinic

Summary Images Lifecycle policy Permissions Repository tags

Images (6) Info

Filter active images

☐	Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
☐	multi	Image index	30 April 2026, 11:55:35 (UTC+05.5)	152.82	sha256:473fc3...	-
☐	-	Image	30 April 2026, 11:55:34 (UTC+05.5)	0.00	sha256:421517...	-
☐	-	Image	30 April 2026, 11:55:34 (UTC+05.5)	152.82	sha256:53efae...	-
☐	latest	Image index	27 April 2026, 18:27:28 (UTC+05.5)	152.82	sha256:76bfdf...	-
☐	-	Image	27 April 2026, 18:27:27 (UTC+05.5)	152.82	sha256:5d4611...	-
☐	-	Image	27 April 2026, 18:27:27 (UTC+05.5)	0.00	sha256:ed001a...	-

PART 3 — Security Scan in ECR

1. Enable Scan on Push

While creating repo:

```
aws ecr put-image-scanning-configuration \  
--repository-name spring-petclinic \  
--image-scanning-configuration scanOnPush=true
```

```
rpavani@C19536 spring-petclinic % aws ecr put-image-scanning-configuration \  
  
--repository-name spring-petclinic \  
  
--image-scanning-configuration scanOnPush=true \  
  
--region eu-north-1
```

```
{
  "registryId": "113304117666",
  "repositoryName": "spring-petclinic",
  "imageScanningConfiguration": {
    "scanOnPush": true
  }
}
```

2. Trigger Scan (if not automatic)

```
aws ecr start-image-scan \
  --repository-name spring-petclinic \
  --image-id imageTag=latest
```

3. Get Scan Report

```
aws ecr describe-image-scan-findings \
  --repository-name spring-petclinic \
  --image-id imageTag=latest
```

4. Analyze Results

```
rpavani@C19536 spring-petclinic % aws ecr describe-image-scan-findings \
  --repository-name spring-petclinic \
  --image-id
imageDigest=sha256:53efae811d08a2f39cf0e45bf244466bc369a74549474ef9a4dac16c1519048b \
  --region eu-north-1
{
  "imageScanFindings": {
    "findings": [
      {
        "name": "CVE-2026-4878",
        "description": "A flaw was found in libcap. A local unprivileged user can exploit a
Time-of-check-to-time-of-use (TOCTOU) race condition in the `cap_set_file()` function. This allows an
attacker with write access to a parent directory to redirect file capability updates to an attacker-controlled
file. By doing so, capabilities can be injected into or stripped from unintended executables, leading to
privilege escalation.",
        "uri": "https://nvd.nist.gov/vuln/detail/CVE-2026-4878",
        "severity": "MEDIUM",
        "attributes": [
          {
            "key": "CVSS3_SCORE",
```

```
        "value": "6.7"
      },
      {
        "key": "CVSS3_VECTOR",
        "value": "CVSS:3.1/AV:L/AC:H/PR:L/UI:R/S:U/C:H/I:H/A:H"
      },
      {
        "key": "package_version",
        "value": "2.44-1ubuntu0.22.04.2"
      },
      {
        "key": "package_name",
        "value": "libcap2"
      }
    ]
  },
  "imageScanCompletedAt": "2026-04-30T11:55:45+05:30",
  "vulnerabilitySourceUpdatedAt": "2026-04-30T11:55:45+05:30",
  "findingSeverityCounts": {
    "MEDIUM": 1
  }
},
"registryId": "113304117666",
"repositoryName": "spring-petclinic",
"imageId": {
  "imageDigest":
"sha256:53efae811d08a2f39cf0e45bf244466bc369a74549474ef9a4dac16c1519048b"
},
"imageScanStatus": {
  "status": "COMPLETE",
  "description": "The scan was completed successfully."
}
}
(END)
```
